ShopLaravel

Rapport Technique

ApplicationE-Commerce complète développée avec le framework Laravel — Architecture MVC

Matiére	Programmation Web 2
Niveau	2ème Année E-Business

oumaima chanbi
ilef sahraoui

Laravel 11 | **PHP 8.3** | **MySQL 8** | **Blade** | **Eloquent ORM** | **Bootstrap 5**

1. Description du Projet

1.1 Présentation générale

ShopLaravel est une plateforme e-commerce complète développée dans le cadre du module **Programmation Web 2**. Elle permet la vente et l'achat de produits en ligne, la gestion des commandes, et l'évaluation des produits par les utilisateurs. L'application est construite sur le framework **Laravel 11**, suivant rigoureusement le patron architectural **MVC** (Model-View-Controller).

■ **Objectif :** Concevoir une application web e-commerce dynamique, multi-utilisateurs, sécurisée et entièrement fonctionnelle en utilisant Laravel comme framework principal, Eloquent ORM pour la base de données, et Blade comme moteur de templates.

1.2 Fonctionnalités principales

■	**Authentification complète**	Inscription, connexion, déconnexion, réinitialisation du mot de passe par email, gestion du profil utilisateur.
■	**Gestion des produits**	CRUD complet (Créer, Lire, Modifier, Supprimer). Chaque utilisateur peut publier ses propres produits avec titre, description, prix, catégorie et image.
■	**Catalogue dynamique**	Recherche par mot-clé, filtrage par catégorie, filtrage par fourchette de prix, tri (date, prix croissant/décroissant), pagination.
■	**Panier d'achat**	Ajout de produits, modification des quantités, suppression d'articles, vidage du panier. Stocké en session Laravel.
■	**Gestion des commandes**	Passage de commande depuis le panier, historique des commandes par utilisateur, affichage du détail, annulation des commandes en attente.
■	**Système d'évaluations**	Note de 1 à 5 étoiles, commentaire textuel, calcul de la note moyenne par produit, suppression de son propre avis.

1.3 Contraintes techniques respectées

■	**Framework obligatoire**	Laravel 11 (PHP 8.3)
■	**Architecture**	MVC stricte — Models, Views (Blade), Controllers
■	**ORM**	Eloquent avec relations hasMany / belongsTo
■	**Base de données**	Migrations + Seeders + Données de test
■	**Sécurité CSRF**	Token @csrf sur tous les formulaires
■	**Sécurité XSS**	Échappement automatique avec {{ }} Blade

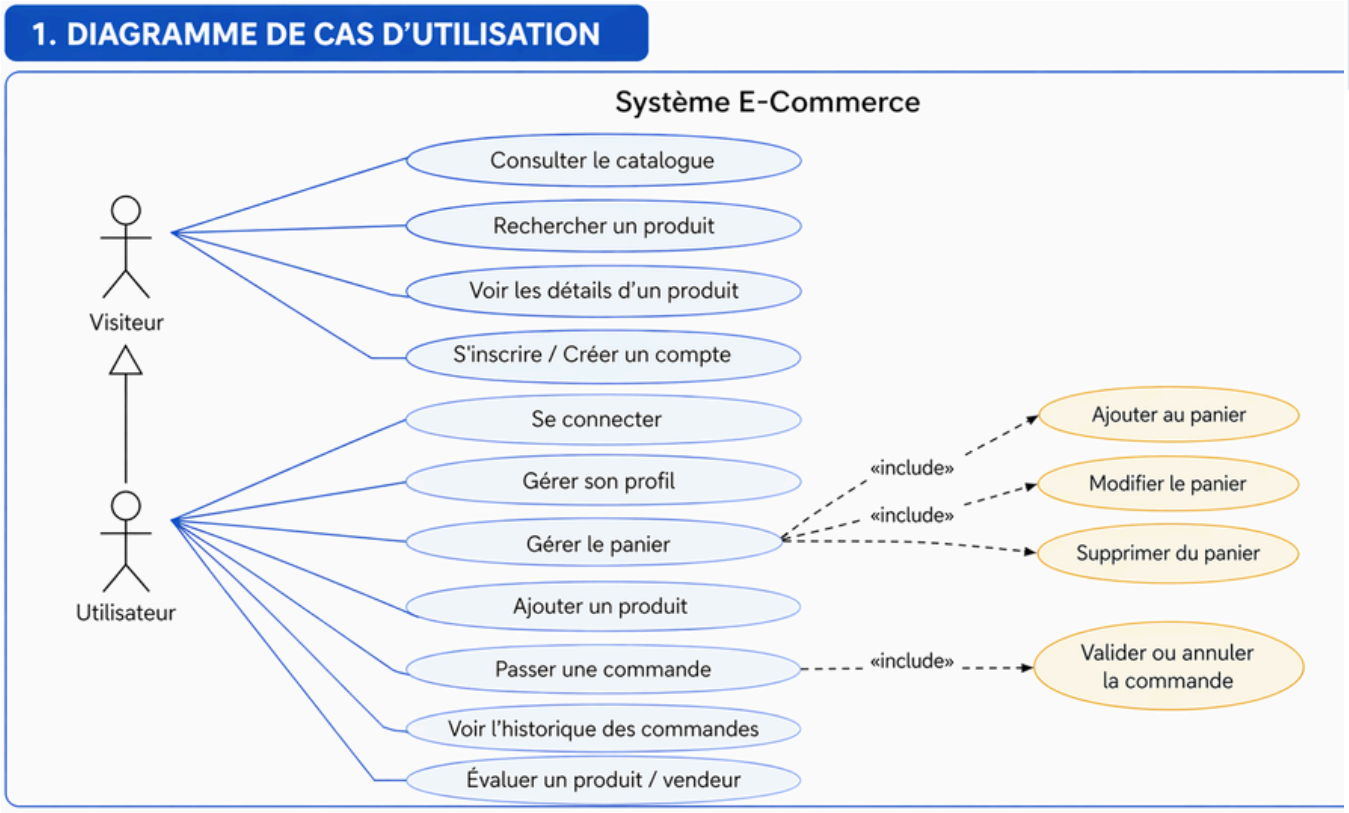
■	**Sécurité SQL**	Requêtes préparées via Eloquent/Query Builder
■	**Autorisations**	Laravel Policies (ProductPolicy, OrderPolicy)
■	**Framework interdit**	Aucun autre framework backend utilisé
■	**CMS interdit**	Pas de WordPress, Drupal ou similaire

2.Description de base de données :

Description des tables:

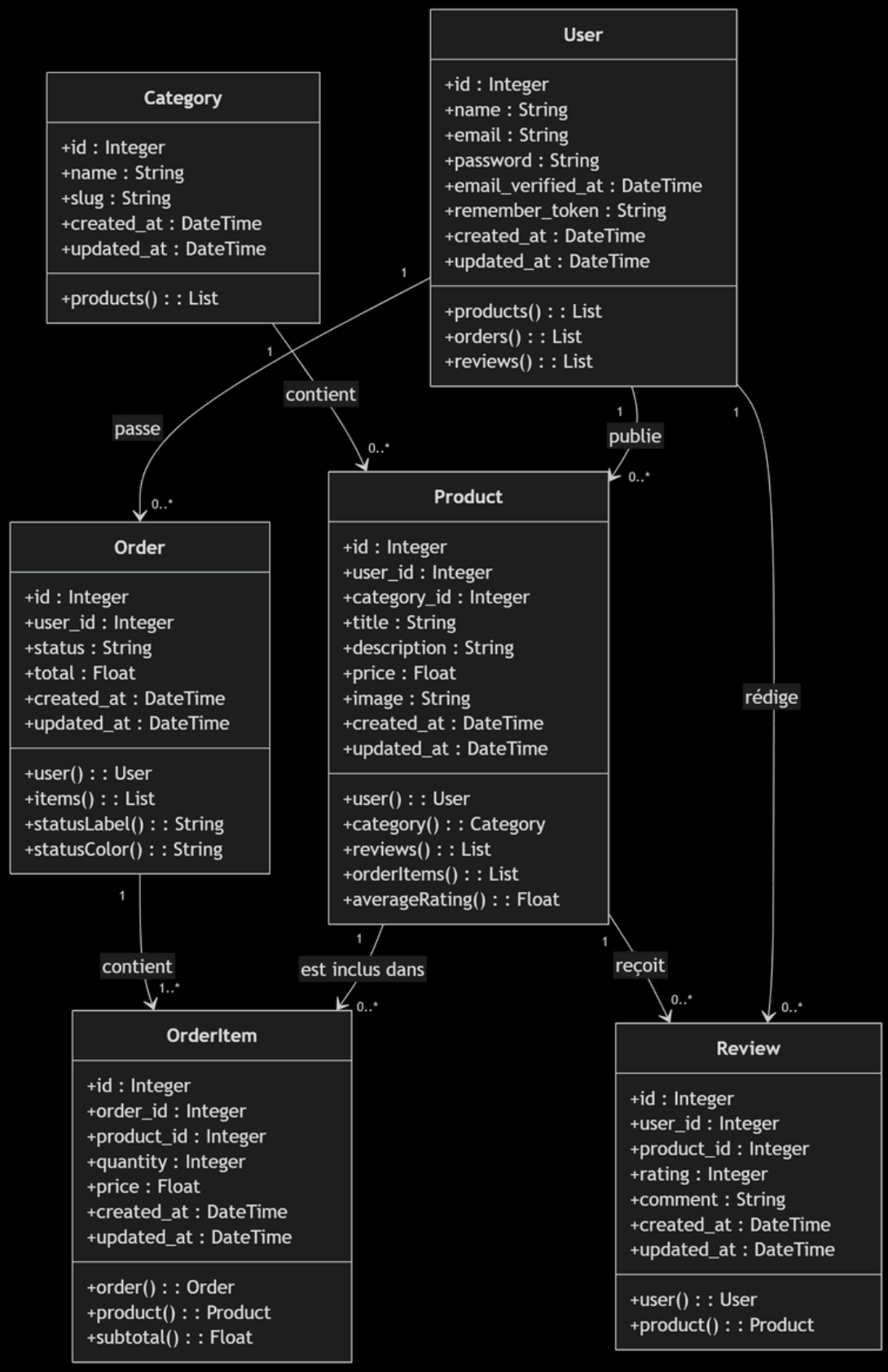
users	Stocke les informations des utilisateurs inscrits. Gérée par Laravel Auth avec hashage bcrypt du mot de passe.
categories	Référentiel des catégories de produits avec un slug unique pour les URLs.
products	Table centrale de la marketplace. Liée à users (vendeur) et categories. Stocke le chemin de l'image uploadée.
orders	Enregistre les commandes avec leur statut (pending/validated/cancelled) et le montant total calculé.
order_items	Table de jonction entre orders et products. Chaque ligne représente un produit dans une commande avec la quantité et le prix au moment de l'achat.
reviews	Évaluations des produits par les utilisateurs. Contrainte UNIQUE sur (user_id, product_id) via updateOrCreate.

■ **Intégrité des données :** Toutes les clés étrangères sont définies avec **onDelete('cascade')** dans les migrations Laravel, ce qui garantit qu'un produit supprimé entraîne automatiquement la suppression de ses évaluations et de ses lignes de commande associées.



3.Description des relations:

User → Product	1..N	Un utilisateur peut publier plusieurs produits (hasMany). Un produit appartient à un seul vendeur (belongsTo).
User → Order	1..N	Un utilisateur peut passer plusieurs commandes. Chaque commande appartient à un utilisateur.
User → Review	1..N	Un utilisateur peut laisser plusieurs avis. Un avis est lié à un seul utilisateur.
Category → Product	1..N	Une catégorie regroupe plusieurs produits. Un produit appartient à une catégorie.
Product → Review	1..N	Un produit peut avoir plusieurs évaluations. Chaque avis concerne un produit.
Product → OrderItem	1..N	Un produit peut apparaître dans plusieurs lignes de commande.
Order → OrderItem	1..N	Une commande est composée de plusieurs articles (lignes de commande).


User
+id : Integer
+name : String
+email : String
+password : String
+email_verified_at : DateTime
+remember_token : String
+created_at : DateTime
+updated_at : DateTime
+products() : : List
+orders() : : List
+reviews() : : List
Category
+id : Integer
+name : String
+slug : String
+created_at : DateTime
+updated_at : DateTime
+products() : : List
passe
contient
publie
rédige
Order
+id : Integer
+user_id : Integer
+status : String
+total : Float
+created_at : DateTime
+updated_at : DateTime
+user() : : User
+items() : : List
+statusLabel() : : String
+statusColor() : : String
Product
+id : Integer
+user_id : Integer
+category_id : Integer
+title : String
+description : String
+price : Float
+image : String
+created_at : DateTime
+updated_at : DateTime
+user() : : User
+category() : : Category
+reviews() : : List
+orderItems() : : List
+averageRating() : : Float
contient
est inclus dans
reçoit
OrderItem
+id : Integer
+order_id : Integer
+product_id : Integer
+quantity : Integer
+price : Float
+created_at : DateTime
+updated_at : DateTime
+order() : : Order
+product() : : Product
+subtotal() : : Float
Review
+id : Integer
+user_id : Integer
+product_id : Integer
+rating : Integer
+comment : String
+created_at : DateTime
+updated_at : DateTime
+user() : : User
+product() : : Product

4. Architecture & Choix Techniques

4.1Structure des fichiers :

`app/Models/`	EloquentModels—User,Product,Category, Order, OrderItem, Review
`app/Http/Controllers/`	HomeController,ProductController , CartController OrderController, ReviewController, ProfileController
`app/Policies/`	ProductPolicy, OrderPolicy — Autorisations basées sur le propriétaire
`database/migrations/`	6 fichiers de migration (schéma de BDD versionné)
`database/seeders/`	UserSeeder, CategorySeeder, ProductSeeder + DatabaseSeeder
`resources/views/`	Vues Blade : layouts/, auth/, products/, cart/, orders/, profile/
`routes/web.php`	Toutes les routes nommées, groupées par domaine fonctionnel
`app/Providers/`	AppServiceProvider — Pagination Bootstrap 5

4.2 Sécurité de l'application

- **Protection CSRF**
  Tous les formulaires POST, PATCH et DELETE incluent la directive **@csrf** de Blade, qui génère un token unique par session. Laravel vérifie automatiquement ce token avant de traiter la requête, empêchant les attaques Cross-Site Request Forgery.

- **Protection XSS**
  Blade échappe automatiquement toutes les variables affichées avec **{{ $var }}** en convertissant les caractères HTML spéciaux (<, >, ", etc.). Aucune donnée utilisateur n'est injectée brute dans le HTML.

- **Protection SQL Injection**
  Eloquent ORM et le Query Builder de Laravel utilisent exclusivement des **requêtes préparées PDO**. Les paramètres utilisateur ne sont jamais concaténés dans les requêtes SQL, éliminant tout risque d'injection.

- **Autorisations (Policies)**
  Les **Policies Laravel** (ProductPolicy, OrderPolicy) vérifient que l'utilisateur authentifié est bien le propriétaire de la ressource avant toute opération de modification ou suppression. Utilisation de **$this->authorize()** dans les controllers.

- **Hachage des mots de passe**
  Laravel utilise **bcrypt** (via Hash::make()) pour stocker les mots de passe. Le cast 'hashed' dans le Model User applique automatiquement le hachage. La comparaison utilise Hash::check() pour la vérification sécurisée.

- **Middleware Auth**
  Les routes sensibles (panier, commandes, profil) sont protégées par le middleware **auth** de Laravel. Les routes de products création/modification utilisent **$this->middleware('auth')** directement dans le constructeur du controller.

4.4 Choix techniques justifiés

Panier en session

Le panier utilise la session Laravel (côté serveur) plutôt qu'une table BDD. Avantage : pas de migration supplémentaire, performances optimales, nettoyage automatique à la déconnexion. La commande convertit la session en entrées persistantes dans orders + order_items.

Bootstrap 5 via CDN

Sans Node.js, Bootstrap est chargé via CDN (jsDelivr). Cela évite toute étape de build (npm run dev) tout en conservant un design responsive et professionnel avec un composant disponible et maintenu.

Pagination Eloquent

La méthode ->paginate(9) d'Eloquent génère automatiquement les liens de pagination Bootstrap 5 grâce à Paginator::useBootstrapFive() dans AppServiceProvider. Simple et efficace.

■ UpdateOrCreate pour les avis

Review::updateOrCreate(['user_id', 'product_id'], [...]) permet à un utilisateur de modifier son avis existant au lieu d'en créer un doublon. Logique métier robuste en une seule ligne Eloquent.

■ Storage::link pour les images

Les images uploadées sont stockées dans storage/app/public/ et accessibles via storage/ grâce au lien symbolique créé par php artisan storage:link. Séparation propre entre fichiers publics et privés.

5. Stack Technique & Conclusion

5.1 Technologies utilisées

Composant	Technologie	Version	Rôle
Backend Framework	Laravel	11.x	MVC, Routing, Auth, ORM
Langage serveur	PHP	8.3	Traitement côté serveur
Base de données	MySQL	8.0	Persistance des données
ORM	Eloquent	Laravel 11	Mapping objet-relationnel
Templates	Blade	Laravel 11	Moteur de rendu des vues
CSS Framework	Bootstrap	5.3	UI responsive via CDN
Icônes	Bootstrap Icons	1.11	Interface utilisateur
Auth	laravel/ui	4.x	Scaffolding authentification
Serveur local	XAMPP/WAMP	—	Apache + MySQL en local

5.2 Checklist des livrables

■	**Fonctionnalités obligatoires**	Authentification, CRUD Produits, Catalogue, Panier, Commandes, Évaluations — Tout implémenté
■	**Architecture MVC**	Models Eloquent, Controllers, Vues Blade, Routes nommées — Structure Laravel respectée
■	**Base de données**	6 tables + migrations + seeders — Données de test complètes
■	**Sécurité**	CSRF + XSS + SQL Injection + Policies — Toutes les protections en place
■	**Interface utilisateur**	Bootstrap 5 responsive, navigation intuitive, alertes dynamiques
■	**Documentation**	Ce rapport technique avec UML, schéma BDD, explication architecture
■	**Bonus niveau 1 (Pagination + Upload)**	Inclus dans le projet (+1 point potentiel)
■	**Bonus niveau 2 (API REST)**	API REST /api/products documentée et implémentable (+2 points potentiels)

5.3 Conclusion

ShopLaravel est une application e-commerce fonctionnelle et complète, développée en respectant l'ensemble des contraintes du cahier des charges. Elle démontre la maîtrise du framework **Laravel 11** à travers une architecture MVC solide, une gestion sécurisée des données, et une interface utilisateur responsive et intuitive.

Les principaux acquis de ce projet incluent la compréhension des **Relations Eloquent**, la gestion des **Middlewares et Policies** pour les autorisations, l'utilisation des **Migrations et Seeders** pour le contrôle de version de la base de données, et l'implémentation de mesures de sécurité robustes contre les vulnérabilités web courantes.

■ **Commandes de démarrage :**
php artisan migrate:fresh --seed | php artisan storage:link | php artisan serve
→ Compte test : admin@test.com / password